

GUI-PROGRAMMERING

Korte svar og refleksioner

De fire GUI-opgaver Dag 1 - 4

1 - GUI-platforme og UI-opbygning

1.1 - Hvad er rendering og delvis rendering?

Rendering omsætter programmets data til det, brugeren ser.

Delvis rendering opdaterer de relevante dele af siden i stedet for at genindlæse hele siden, fx når vi vælger en ny tabel i dette projekt.

1.1 - Hvad er render-tree og DOM?

Render-tree er Blazors interne beskrivelse af brugerfladen.

DOM er browserens struktur af HTML-elementer. Blazor sammenligner den nye beskrivelse med den forrige og sender de nødvendige ændringer til browseren.

1.2 - Hvilken teknologi bruger vi?

Vi bruger Blazor, som er brugt til SPA's og derfor har indbyggede delvise rendering - via. InteractiveServer.

C# koden kører på serveren, mens browseren viser siden og sender brugerens handlinger tilbage. Derfor behøver vi ikke selv at bruge fetch, jQuery eller HTMX til tabelskift.

I koden:

Components/App.razor: <Home @rendermode="InteractiveServer" />.

1.2 - Hvordan vises tabellerne?

Home henter tabelnavnene fra SQL Server og viser dem i en dropdown.

Det valgte navn sendes som TableName til DatabaseTable.

Komponenten henter tabellens data og bruger @foreach til at vise kolonner og rækker.

I koden:

Components/Pages/Home.razor: OnInitialized og selectedTable.

Components/Pages/PartialView/DatabaseTable.razor: LoadTable.

DBAccess/Repository.cs: GetTableNames og GetTable.

1.3 - Hvordan hænger den delvise rendering sammen?

@bind ændrer selectedTable, når brugeren vælger i dropdownen.

Derefter renderer Blazor igen, og @key="selectedTable" får den gamle DatabaseTable erstattet af en ny.

Den nye komponent henter data og starter på side 1; hele websiden genindlæses ikke.

I koden:

Components/Pages/Home.razor: <DatabaseTable @key="selectedTable" ... />.

@key er komponentens identitet hos Blazor. Den er ikke en databaseforespørgsel eller en primærnøgle i SQL Server.

1 - Sprogvalg og state

2.1 - Hvor er sprogmenuen og billedet?

LanguageSelector vises i toolbaren øverst til højre i Home.

Komponenten indeholder et globusbillede og en dropdown med Dansk og English.

I koden:

Components/Pages/Home.razor: header.

Components/Pages/CustomControls/LanguageSelector.razor.

wwwroot/images/language.svg.

2.2 - Hvordan husker siden sproget?

UiState gemmer sproget som enumtypen Language.

Klassen registreres som scoped, så hver interaktiv Blazor-forbindelse har sin egen tilstand.

Home binder sprogmenuen til State.Language og sender sproget videre til tabelkomponenten.

I koden:

Program.cs: AddScoped<UiState>().

State/UiState.cs.

Components/Pages/Home.razor: @bind-Language="State.Language".

2.2 - Hvorfor forstyrres tabellen ikke ved sprogskit?

Sprogskit ændrer Language, men ikke selectedTable eller tabellens @key.

Derfor bliver den samme tabelkomponent og dens aktuelle side bevaret.

Ved tabelskift oprettes en ny tabelkomponent, mens sproget fortsat ligger i UiState.

2.3 - Delvis rendering med og uden session/state

Delvis rendering beskriver, hvordan visningen opdateres.

State beskriver de værdier, programmet husker mellem handlingerne.

Først styrer selectedTable den viste tabel; med UiState huskes sprogvalget også, når en anden tabel vælges.

UiState er tilstand i Blazor-forbindelsen, ikke permanent lagring eller ASP.NET Session. En fuld sidegenindlæsning starter normalt med standardsproget dansk.

Hvordan virker @inject og de fælles oversættelser?

@inject beder Blazors servicecontainer om et registreret objekt.

Home får UiState, og databasekomponenterne får Repository.

Teksterne findes i Translations.cs, hvor switch-udtryk vælger teksten ud fra Language.

I koden:

Program.cs: AddDbContextFactory, AddScoped<Repository> og AddScoped<UiState>.

State/Language.cs.

Translations.cs.

Kort demonstration

Vælg Product og gå til side 2.

Skift til English: titlen, sideinformationen og knapperne skifter sprog, mens tabellen bliver på side 2.

Vælg derefter en anden tabel: den starter på side 1, og sproget er stadig engelsk.

2 - Interaktion og UI-arkitektur

1.1 - Hvordan opretter, redigerer og sletter vi?

Ny post åbner RecordEditor med felter til en ny række.
Blyanten åbner samme formular med den valgte rækkes værdier.
Ved redigering kan man også vælge Slet og derefter bekræfte sletningen.

I koden:

Components/Pages/PartialView/DatabaseTable.razor: OpenEditor.

Components/Pages/PartialView/RecordEditor.razor: Save og Delete.

1.2 - Hvad er event-handling i vores løsning?

Et event er fx et klik eller indsendelse af en formular.
Event handleren er metoden, der reagerer: @onclick kalder OpenEditor, og formularens @onsubmit kalder Save. Metoden ændrer data eller tilstand, og Blazor opdaterer visningen.

1.2 - Hvilket UI-designmønster bruger vi?

Vi bruger Modal Dialog: et vindue oven på den aktuelle side.
Den samme formular bruges på tværs af tabeller til oprettelse og redigering, og sletning sker fra redigeringsvinduet.
Det giver en ensartet arbejdsgang uden at forlade tabellen.

I koden:

Components/Pages/PartialView/RecordEditor.razor: <dialog>.

Components/App.razor: showRecordEditor.

1.2 - Hvordan bliver ændringerne gemt og vist?

Repository.SaveChanges bruger rækkens tilstand til at udføre INSERT, UPDATE eller DELETE med SqlDataAdapter og SqlCommandBuilder.
Efter en vellykket handling sender RecordEditor besked via Saved.
DatabaseTable genindlæser data og giver Home besked, så rækkeantallene også opdateres.

I koden:

DBAccess/Repository.cs: SaveChanges.

Components/Pages/PartialView/DatabaseTable.razor: RecordSaved og DataChanged.

Components/Pages/Home.razor: RefreshOverview.

Hvordan undgår vi ændringer af ID og låste felter?

Eksisterende primærnøgler, automatisk tildelte ID'er og andre skrivebeskyttede felter låses i formularen.
Save kontrollerer også låste værdier på serveren.
Formularen arbejder med en kopi, så Annuller ikke gemmer de indtastede ændringer.

I koden:

Components/Pages/PartialView/RecordEditor.razor: IsReadOnly, originalValues og Save.

DBAccess/Repository.cs: MissingSchemaAction.AddWithKey henter også oplysninger om primærnøglen.

Hvordan bevares sproget, og hvad viser vi ved fejl?

CRUD ændrer ikke UiState.Language, så sprogvalget bevares.
Gemte ændringer får en statusbesked.
Hvis databasen afviser gemning eller sletning, bliver formularen stående med en fejlbesked, så brugeren kan rette eller annullere.

I koden:

Components/Pages/PartialView/DatabaseTable.razor: changesSaved.

Components/Pages/PartialView/RecordEditor.razor: errorMessage og catch.

Translations.cs: ChangesSaved, SaveError og DeleteError.

3 - Avanceret GUI-programmering

1 - Hvad er lifecycle/lifetime-events?

Det er tidspunkter i en applikations eller komponents levetid, hvor kode kan køre. I vores Blazor-app registrerer Program.cs services og starter appen. Komponenterne bruger lifecycle-metoder til fx at hente data, når de oprettes.

I koden:

Program.cs: builder.Services og app.Run().

Components/Pages/Home.razor: OnInitialized.

Components/Pages/CustomControls/DBService.razor: OnInitialized.

Components/Pages/PartialView/DatabaseTable.razor: OnInitialized.

Hvad betyder de nævnte lifecycle-metoder?

OnInitialized bruges ved initialisering.

OnParametersSet bruges, når komponentens parametre sættes.

OnAfterRender bruges efter rendering, hvor HTML-elementer kan tilgås.

Dispose bruges til oprydning, hvis komponenten har ressourcer eller abonnementer, der skal afsluttes.

Hvorfor åbnes dialogen i OnAfterRenderAsync?

Dialogens HTML-element skal først eksistere i browseren.

Ved firstRender kalder RecordEditor derfor JavaScript-funktionen showRecordEditor via await JS.InvokeVoidAsync. firstRender sikrer, at kaldet kun sker ved komponentens første interaktive rendering.

I koden:

Components/Pages/PartialView/RecordEditor.razor: OnAfterRenderAsync.

Components/App.razor: showRecordEditor.

2, punkt 1.1 - Hvad gør vores custom control?

DBService er en komponent, vi selv har lavet.

Den henter aktuelle tabelnavne og antal rækker fra databasen og viser en liste.

Home placerer den til venstre for tabelvisningen, så den stadig vises, når brugeren skifter tabel.

I koden:

Components/Pages/CustomControls/DBService.razor: LoadTable.

DBAccess/Repository.cs: GetTableNames og GetRecordCount.

2, punkt 1.2 - Hvordan opdateres den automatisk?

Efter en vellykket CRUD-handling kalder DatabaseTable DataChanged.

Home øger overviewVersion, som bruges som @key på DBService.

Blazor opretter derfor komponenten igen, og OnInitialized henter de nye rækkeantal.

2, punkt 1.2 - Har vi asynkron kørsel?

JavaScript-kaldet til dialogen er asynkront, og EventCallback.InvokeAsync returnerer en Task, som kan afventes.

Det starter ikke i sig selv en ny tråd.

Databasens Fill, ExecuteScalar og Update er stadig synkrone; DBService er derfor ikke en baggrundsproces.

Nuværende status: Rækkeantal opdateres efter appens egne CRUD-handlinger. Opgavens baggrundskørsel, et særskilt samlet tabelantal og løbende registrering af ændringer uden for appen er ikke implementeret. De dele er dermed ikke dokumenteret som opfyldt.

4 - Usability og evaluering

Hvad er usability, og hvem er målgruppen?

Usability handler om, hvor let, effektivt og tilfredsstillende det er at bruge systemet. Målgruppen er almindelige brugere, som skal oprette, læse, redigere og slette oplysninger. De skal kunne udføre arbejdet uden at kende SQL.

1 - Hvordan er projektet struktureret?

CustomControls og PartialView ligger under Components/Pages.
DBService og LanguageSelector ligger i CustomControls.
DatabaseTable og RecordEditor ligger i PartialView.

```
Components/Pages/  
  Home.razor  
  CustomControls/  
    DBService.razor  
    LanguageSelector.razor  
  PartialView/  
    DatabaseTable.razor  
    RecordEditor.razor
```

1 - Er sprogmenuen en custom component eller et partial view?

Jeg har valgt at gøre sprogmenuen til en custom component.
LanguageSelector samler ikon, dropdown og beskeden om sprogændring i én genbrugelig kontrol. Den modtager Language og sender et nyt valg tilbage med LanguageChanged.

I koden:

Components/Pages/CustomControls/LanguageSelector.razor: ChangeLanguage.

Components/Pages/Home.razor: <LanguageSelector @bind-Language="State.Language" />.

Hvad er forskellen på rollerne i Blazor?

Både vores custom controls og partial views er teknisk Razor-komponenter.
Custom control beskriver her en genbrugelig kontrol med en afgrænset funktion.
Partial view beskriver en del af sidens indhold, fx tabellen eller formularen; det er ikke et separat MVC-partial view.

2 - Hvordan kan brugertesten gennemføres?

Ved hentning af projektet.

4 - Svar på de fem spørgsmål

Svarene nedenfor er en vurdering af den nuværende løsning.

Lærbarhed - Kan en ny bruger hurtigt forstå systemet?

Visuelt simpelt. Ny post og den fælles formular giver et ensartet forløb.

Blyanten for redigerings symbol, med en forklarende tekst ved musemarkering.

Effektivitet - Kan brugeren løse opgaven hurtigt?

Ja. Opret og rediger kræver få klik, og brugeren bliver på samme side.

Sideinddelingen begrænser visningen til 100 rækker ad gangen.

At finde en bestemt post i en stor tabel kan dog tage tid, fordi der ikke er en søgefunktion.

Fejl - Hvor ofte laver brugeren fejl?

Der var ikke nogen oplyste, oplevede fejl.

Låste ID'er, obligatoriske felter og bekræftelse før sletning hjælper med at forebygge fejl.

Indtastning af fremmednøgle-ID'er kan stadig føre til afviste ændringer.

Hukommelse - Kan brugeren finde ud af systemet igen efter en pause?

De samme knapper og den samme formular bruges på tværs af tabeller, hvilket kan gøre arbejdsgangen lettere at huske.

Tilfredshed - Føles systemet logisk og behageligt at bruge?

Den fælles formular, Annuller og beskeden om gemte ændringer giver et tydeligt forløb.

Det samlede resultat på 4,5 tyder på en positiv oplevelse.

Der er ikke oplyst en særskilt karakter for tilfredshed eller konkrete kommentarer.

Samlet resultat og refleksion

Testpersonernes oplyste gennemsnit: 4,5 ud af 5.

Jeg antager at 1 = dårligt og 5 = godt.